Last Class

1, Strategy to reduce global memory traffic

2, Tiled Matrix-matrix multiplication kernel

3, Analyzing how much global memory access is reduced for the new kernel?

Today

1, Reduction in parallel programming (first)

2, Memory as a limiting factor to parallelism (Second)

Reduction in GPU programming

- Definition of Reduction in parallel computing
  - A reduction algorithm derives a single value from an array of input values.
  - The single value could be sum, max, min, product, among all elements.
  - All these types of reductions share the same computation structure,
    - one of popular parallel paradigms(primitives).
  - Notices that, the operations that reduction use, (sum, max, min) are **associative** and **commutative**.

Reduction in GPU programming

- The next few pdf slides are based on Mark Harris's presentation from Nvidia.

Memory as a Limiting Factor

- CUDA device has limited amount of resources
  - Limited size of shared memory per block
  - Limited # of register per SM
  - Limited # of threads simultaneously residing in SM
  - Limited # of blocks assigned to a SM

- If a thread needs more resources,
  - Then fewer threads can reside in each SM
  - Then fewer threads can reside in the entire device.

Memory as a Limiting Factor

- Example Analysis
- Assume we have a device, (queryDevice returns these limitations)
  - Each SM allows maximum of 1536 threads residing in device at the same time.
  - Each SM has 16384 registers
- To support 1536 threads, (reach its max capacity),
  - Each thread can use only 10 registers (16384 / 1536 = 10.6).

Memory as a Limiting Factor

- Example Analysis
- What if each thread use 11 registers?
  - # of threads able to be executed concurrently in each SM is reduced.
  - Note that this reduction is done at the block granularity. (threads assigned to SM on a per block basis)
  - That is, if block size is 512 threads, reduction of threads will be done by reducing 512 threads at a time.
    - This is one-third of the original # of threads in SM.

Memory as a Limiting Factor

- Example Analysis
- This reduction greatly reduce the number of warps available for scheduling,
- In turn, reduce the SM's ability to find useful work in presence of long-latency operations.
  - We learned this before when discussing latency hiding.

Memory as a Limiting Factor

- Example Analysis 2
- The same device also has a limited size of shared memory in each SM, assuming it is 16KB.
- Assume each SM can accommodate up to 8 blocks.
- Then each block must NOT use more than 2 KB of shared memory, in ORDER TO achieve this max capacity.
- What if each block uses more than 2 KB of shared memory in this case?

Memory as a Limiting Factor

- Example Analysis 2
- What if each block uses more than 2 KB of shared memory in this case?
- The # of blocks that can reside in SM simultaneously is reduced, such that the total amount of shared memory used is <= 16KB.
- E.g. if each block use 5KB of shared memory, no more than 3 blocks assigned to each SM.

Tiled Matrix-Matrix Multiplication

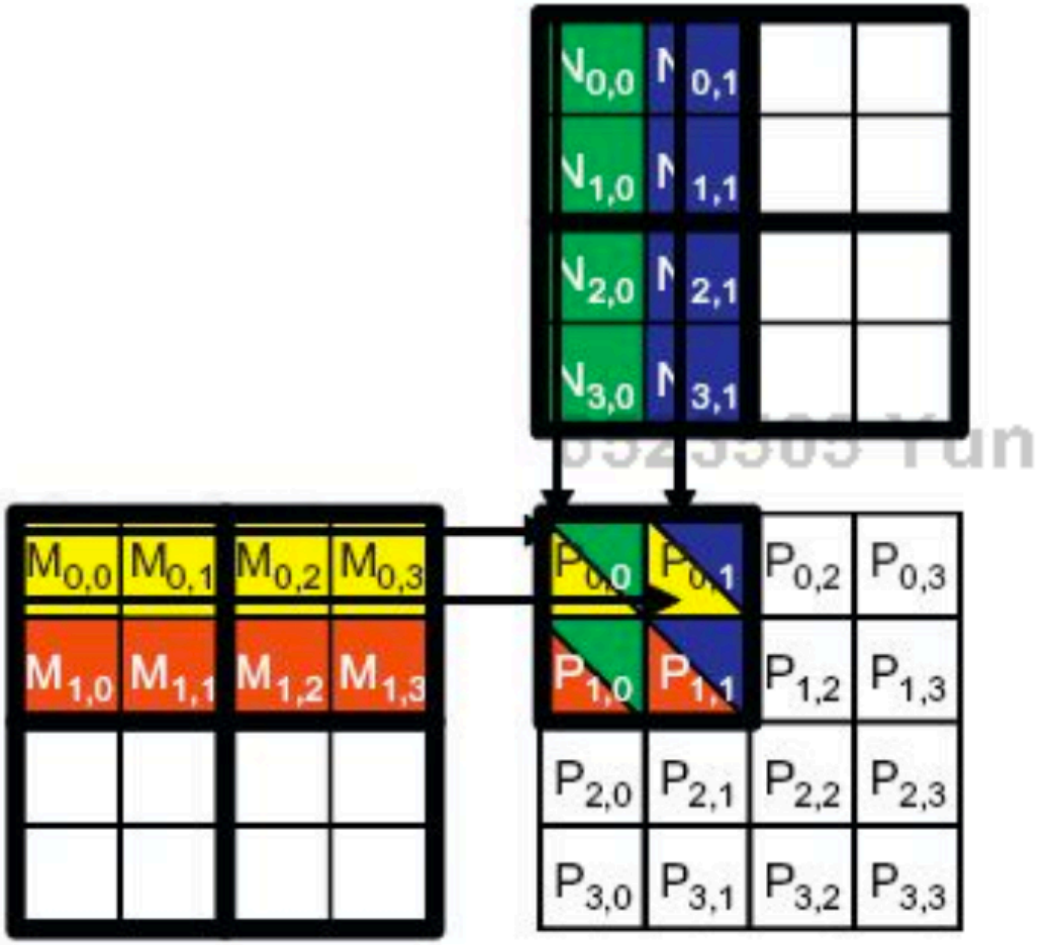


FIGURE 5.10

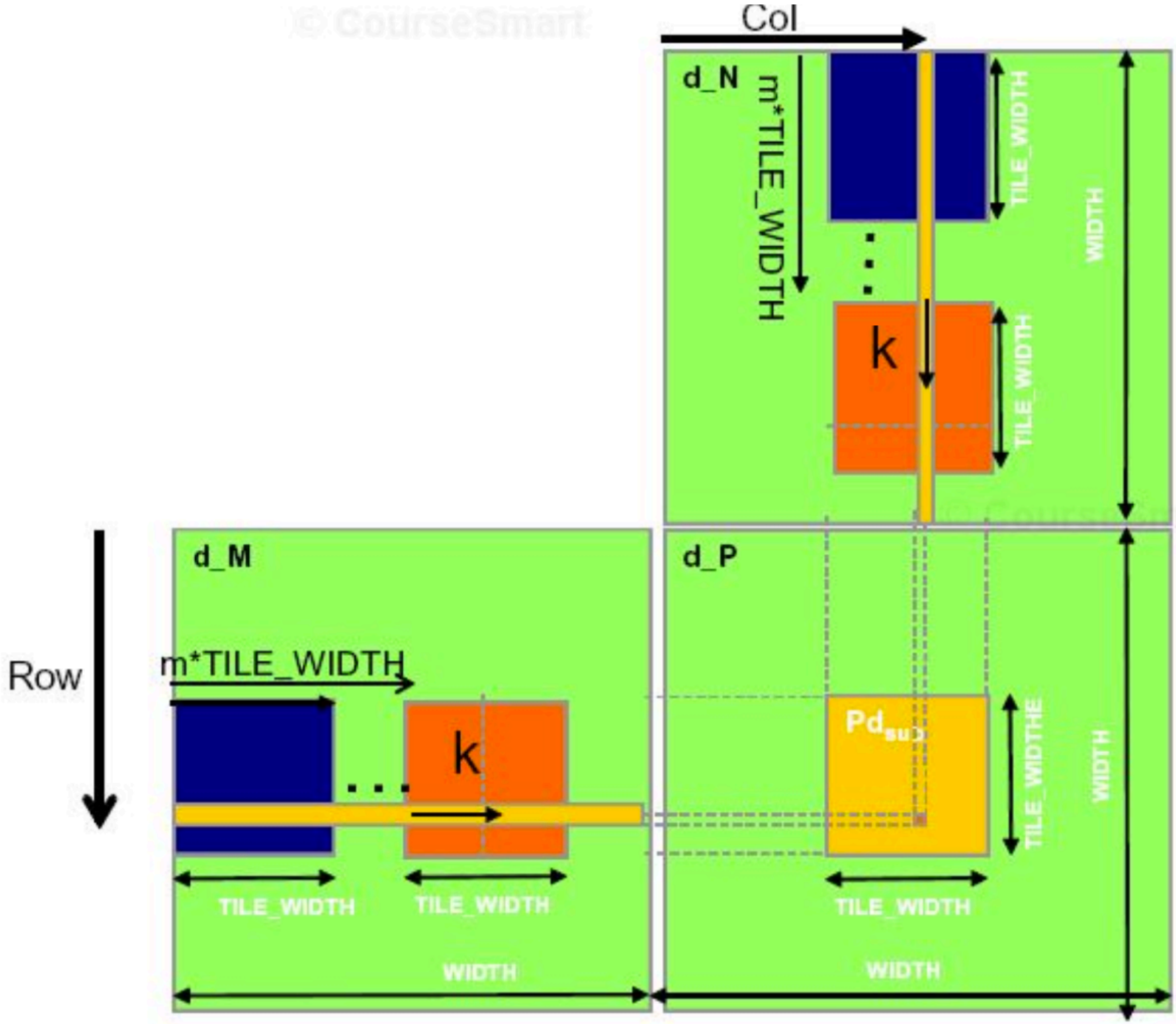
Tiling M and N matrices to utilize shared memory.

```
__global__ void MatrixMulKernel(float* d_M, float* d_N, float* d_P,
     int Width) {

  __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
  __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];

  int bx = blockIdx.x;  int by = blockIdx.y;
  int tx = threadIdx.x; int ty = threadIdx.y;

  // Identify the row and column of the d_P element to work on
  int Row = by * TILE_WIDTH + ty;
  int Col = bx * TILE_WIDTH + tx;

  float Pvalue = 0;
  // Loop over the d_M and d_N tiles required to compute d_P element
  for (int m = 0; m < Width/TILE_WIDTH; ++m) {

    // Coolaborative loading of d_M and d_N tiles into shared memory
    Mds[ty][tx] = d_M[Row*Width + m*TILE_WIDTH + tx];
    Nds[ty][tx] = d_N[(m*TILE_WIDTH + ty)*Width + Col];
    __syncthreads();

    for (int k = 0; k < TILE_WIDTH; ++k) {
      Pvalue += Mds[ty][k] * Nds[k][tx];
    }
    __syncthreads();
  }
  d_P[Row*Width + Col] = Pvalue;
}
```

FIGURE 5.12

Tiled matrix multiplication kernel using shared memory.

© CourseSmart



Memory as a Limiting Factor

- Example Analysis 3
- In matrix-matrix multiplication kernel in last slide, shared memory could be a limiting factor.
  - If the tile size of 16 X 16, each block needs 16 * 16 * 4 = 1KB of storage of Mds, and another 1KB for Nds.
  - Then each block uses 2KB of shared memory.
  - The 16 KB shared memory allows 8 blocks assigned to each SM. (8 is maximum blocks that can reside in SM simultaneously)

Memory as a Limiting Factor

- Example Analysis 3
- In matrix-matrix multiplication kernel in last slide, shared memory could be a limiting factor.
  - Remember that each device also has limit on the # of threads that can simultaneously reside in each SM.
    - Assume for this device is 1536.
  - Then 1536 / (16 * 16) = 6. Only at most 6 blocks actually can reside in a SM at one time.
  - Then the total amount of shared memory that will be used in this case is 6 * 2KB = 12KB. (< 16KB the max size)

Wrap up

1, One basic Reduction kernel

2, Memory as limiting factor for parallelism

Next Class

1, Performance considerations